

GRAPHIC ERA HILL UNIVERSITY

Dehradun, Uttarakhand

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

FINAL YEAR MAJOR PROJECT REPORT

TITLE :-

Crowd Safety and Management System

B.TECH (CSE)

VIII SEMESTER 2025-26

Submitted By: NAME : Ayush Nainwal UNI ROLL NO. / SECTION : 2218571/ C2 CONTACT NO. : 7017622541 NAME : Mayank Chandel UNI ROLL NO. / SECTION : 2219065 / E2 CONTACT NO. : 9528234491 NAME : Ayush Garg UNI ROLL NO. / SECTION : 2218558/ A2 CONTACT NO. : 7409130112 NAME : Himanshu Negi UNI ROLL NO. / SECTION : 2218854/ A2 CONTACT NO. : 9760556657	Submitted To : Manisha Aeri Department of CSE Graphic Era Hill University
---	---

CERTIFICATE

This is to certify that the project report entitled "**Crowd Safety and Management System**" submitted by **Ayush Nainwal (2218571)**, **Mayank Chandel (2219065)**, **Ayush Garg (2218558)**, and **Himanshu Negi (2218854)** of B.Tech (CSE), VIII Semester, is a bona fide work carried out by them in the Department of Computer Science and Engineering, Graphic Era Hill University, Dehradun, under the supervision and guidance of **Ms. Manisha Aeri**, during the academic year 2025-26.

This project fulfils the requirements for the partial completion of the degree of Bachelor of Technology in Computer Science and Engineering as prescribed by Graphic Era Hill University. The work embodied in this report has not been submitted for the award of any other degree or diploma.

Ms. Manisha Aeri Project Supervisor Department of CSE Graphic Era Hill University	Head of Department Department of CSE Graphic Era Hill University
---	---

ACKNOWLEDGEMENT

We express our sincere gratitude to Ms. Manisha Aeri, Department of Computer Science and Engineering, Graphic Era Hill University, for her invaluable guidance, constant encouragement, and constructive feedback throughout the development of this project. Her expert knowledge and dedication have been instrumental in shaping this work.

We are deeply thankful to the Head of Department and all the faculty members of the Department of Computer Science and Engineering for providing us with the necessary resources, infrastructure, and academic support to successfully complete this project.

We extend our heartfelt thanks to the Graphic Era Hill University management for providing an outstanding academic environment and the technical infrastructure required for conducting research and development activities.

We also acknowledge the contributions of the open-source community, particularly the developers of YOLOv8 (Ultralytics), OpenCV, Flask, and the numerous Python libraries that formed the backbone of this system. The publicly available datasets including ShanghaiTech and CrowdHuman proved invaluable during model training and evaluation.

Finally, we are grateful to our families and friends for their continuous support, motivation, and understanding during the course of this project. Their encouragement kept us going during the challenging phases of development.

Ayush Nainwal
Mayank Chandel
Ayush Garg
Himanshu Negi

ABSTRACT

The rapid growth of urban populations and the increasing frequency of large-scale public gatherings have made crowd safety a critical concern worldwide. Incidents of stampedes, injuries, and fatalities due to unmanaged crowd density necessitate intelligent and automated crowd monitoring solutions that can operate in real time and provide timely warnings to authorities.

This project presents the design, development, and implementation of a Crowd Safety and Management System built using Python, deep learning, and web technologies. The system employs YOLOv8, a state-of-the-art object detection model, to detect and count individuals in live video streams captured via cameras. Detected persons are analyzed for spatial distribution, enabling the system to compute crowd density, identify congestion hotspots, and classify risk levels into Low, Medium, and High categories.

A critical aspect addressed in this report is the methodology of density calculation. The system evolves from a naive pixel-space approach (counting people per frame area) to a grid-based spatial analysis (4x4 grid hotspot detection), and finally to a real-world calibrated approach using OpenCV homography. The homography-based method maps pixel coordinates to actual metric ground coordinates using four reference points, enabling accurate calculation of crowd density in people per square metre as per the internationally recognized Fruin Level of Service scale.

The system incorporates heatmap visualization for intuitive density representation, a Flask-based real-time web dashboard for monitoring, an automated email alert system for high-risk notifications, and optional IoT integration via Arduino for physical response mechanisms. The modular architecture ensures scalability for multi-camera deployment and cloud integration.

Experimental results demonstrate reliable real-time detection performance, accurate risk classification, effective heatmap visualization, and successful alert delivery. The system is suitable for deployment at railway stations, shopping malls, university campuses, religious gatherings, and other large public venues, contributing significantly to proactive crowd management and public safety.

Keywords: *Crowd Safety, YOLOv8, Object Detection, Crowd Density Estimation, Homography, Heatmap Visualization, Real-Time Monitoring, Flask, Risk Assessment, Computer Vision, Deep Learning.*

TABLE OF CONTENTS

Certificate.....	i
Acknowledgement	ii
Abstract.....	iii
Table of Contents.....	iv
List of Figures	v
List of Tables	vi
Chapter 1: Introduction	1
1.1 Background	1
1.2 Problem Statement	3
1.3 Objectives of the Project	4
1.4 Scope of the Project	5
1.5 Organization of the Report.....	6
Chapter 2: Literature Review	7
2.1 Traditional Crowd Monitoring Methods.....	7
2.2 Deep Learning-Based Approaches.....	8
2.3 YOLO-Based Detection Systems.....	9
2.4 Density Estimation Techniques	10
2.5 Existing Systems and Research Gaps	11
Chapter 3: System Design and Architecture	13
3.1 System Overview	13
3.2 Module-Level Architecture.....	14
3.3 Data Flow Diagram.....	16
3.4 Technology Stack.....	17
Chapter 4: Crowd Density Calculation — Methodology and Analysis	20
4.1 The Core Problem: What is Density?	20
4.2 Approach 1 — Frame-Level Count (Naive Baseline).....	21
4.3 Approach 2 — Grid-Based Spatial Analysis	22
4.4 Approach 3 — Homography-Based Real-World Calibration	24
4.5 Fruin Level of Service Scale.....	27
4.6 Comparison of Approaches.....	28
Chapter 5: Implementation	30
5.1 Development Environment Setup	30
5.2 YOLOv8-Based Object Detection Module.....	31
5.3 Heatmap Generation Module	33
5.4 Risk Assessment Module	35
5.5 Web Dashboard (Flask)	36
5.6 Email Alert Module	38
5.7 IoT Integration (Arduino)	39
Chapter 6: Testing and Results	41
6.1 Testing Methodology	41
6.2 Detection Performance.....	42
6.3 Density Calculation Accuracy	43

6.4 Alert System Testing.....	44
6.5 Dashboard Testing	45
Chapter 7: Advantages, Limitations, and Future Scope.....	46
7.1 Advantages.....	46
7.2 Limitations	47
7.3 Future Scope	48
Chapter 8: Conclusion.....	51
References.....	52
Appendix A: Source Code	54

LIST OF FIGURES

Figure 1.1: Crowd gathering at a public event.....	2
Figure 1.2: Challenges in manual crowd monitoring.....	3
Figure 3.1: Overall system architecture — Stampede Prevention System	14
Figure 3.2: Module-level data flow in the system	16
Figure 3.3: Project folder structure	18
Figure 4.1: Illustration of pixel-density flaw with different camera heights	21
Figure 4.2: 4x4 grid spatial analysis with hotspot detection.....	23
Figure 4.3: Homography point mapping — pixel to real-world coordinates.....	25
Figure 4.4: Fruin Level of Service pedestrian density zones	27
Figure 5.1: YOLOv8 detection pipeline with bounding boxes	32
Figure 5.2: Heatmap overlay on live video frame.....	34
Figure 5.3: Risk level classification threshold visualization.....	35
Figure 5.4: Flask web dashboard — real-time monitoring interface	37
Figure 5.5: Email alert sample — Crowd Risk Alert.....	38
Figure 5.6: Arduino IoT integration schematic.....	40
Figure 6.1: Detection accuracy across different crowd densities.....	42
Figure 6.2: Risk level transitions over time	44

LIST OF TABLES

Table 2.1: Comparison of crowd monitoring techniques.....	12
Table 3.1: Technology stack and purpose.....	19
Table 4.1: Comparison of density calculation approaches.....	28
Table 4.2: Fruin Level of Service classification	27
Table 5.1: Risk level thresholds.....	35
Table 6.1: Detection performance metrics.....	43
Table 6.2: Alert response time measurements	45
Table 7.1: Advantages and corresponding system features	46

CHAPTER 1: INTRODUCTION

1.1 BACKGROUND

The rapid urbanisation of modern societies and the growing frequency of large-scale public events have collectively transformed crowd management into one of the most pressing challenges of the twenty-first century. Every year, millions of people gather at railway stations, airports, religious pilgrimage sites, sporting arenas, music concerts, and public festivals. While these gatherings are a testament to the vibrancy of human social life, they also pose substantial risks when crowd density exceeds safe thresholds.

Historical incidents have demonstrated, time and again, the catastrophic consequences of overcrowding. The 1990 Mecca tunnel stampede, the 2010 Love Parade disaster in Duisburg, the 2015 Hajj stampede, and the 2022 Halloween crowd crush in Seoul are grim reminders that crowd safety cannot be left to chance or manual supervision alone. In India, stampedes at Kumbh Mela, railway stations, and temple premises have claimed hundreds of lives, underscoring the urgent need for automated crowd safety mechanisms.

Traditional approaches to crowd monitoring rely heavily on human operators watching surveillance camera feeds, security personnel conducting physical patrols, and post-incident analysis. These methods suffer from critical limitations: humans become fatigued, have limited field of vision, cannot process multiple camera feeds simultaneously, and are inherently reactive rather than proactive. By the time a security officer recognises a dangerous crowd buildup and communicates it up the chain of command, the situation may already have escalated beyond control.

Advances in artificial intelligence, particularly in the domains of computer vision and deep learning, have opened new possibilities for automated crowd analysis. Modern object detection algorithms can process video frames in real time, detect and count individuals with high accuracy, and provide spatial analysis of crowd distribution — all without human fatigue or attention limitations. These capabilities form the technological foundation of this project.

Figure 1.1: Representative image of crowd density at a large public gathering, illustrating the need for automated monitoring

The Crowd Safety and Management System developed in this project integrates YOLOv8-based person detection, heatmap-based density visualisation, a Flask web dashboard, and an automated email alert system into a cohesive real-time crowd safety platform. A key contribution of this work is a thorough analysis of density calculation methodologies — from naive pixel-space approaches to calibrated real-world metric computation using homography — providing both theoretical insight and practical implementation.

1.2 PROBLEM STATEMENT

Managing crowd safety in real time presents a multi-dimensional challenge. The absence of automated monitoring systems capable of detecting overcrowding before it escalates into dangerous situations means that human lives remain at risk at large public venues every day. Manual monitoring methods are demonstrably insufficient for large-scale environments and do not provide timely warnings.

A critical technical problem that existing systems often overlook is the accurate calculation of crowd density. The most naive approach — simply counting persons detected in a video frame — is fundamentally flawed. The detected count reflects the number of individuals visible in the camera frame, but that frame may cover an area of ten square metres or fifty square metres depending on the camera's height, angle, and focal length. A count of thirty people crammed into ten square metres represents a life-threatening emergency. The same thirty people spread across fifty square metres may be completely safe. Without knowing the real-world area captured by the frame, the raw count is meaningless as a density metric.

Figure 1.2: Illustration of monitoring limitations — same crowd count, vastly different real-world density based on camera parameters

The specific problems addressed in this project include the inability to monitor crowd density continuously, the lack of real-time spatial risk assessment, the absence of intelligent visualisation tools for identifying congestion hotspots, delayed communication of warnings to authorities, and the heavy reliance on human supervision that characterises existing deployments.

1.3 OBJECTIVES OF THE PROJECT

The primary objectives of the Crowd Safety and Management System are enumerated below:

- To design and implement a real-time crowd monitoring system using computer vision and deep learning techniques.
- To detect and accurately count persons from live video streams captured via camera feeds.
- To analyse crowd density dynamically on a per-frame basis using three progressively refined approaches — frame-level counting, grid-based spatial analysis, and homography-calibrated real-world metric computation.
- To classify crowd risk levels into Low, Medium, and High categories based on configurable thresholds derived from the Fruin Level of Service scale.
- To generate heatmap visualisations that intuitively highlight high-density zones within the monitored area.
- To develop a web-based real-time dashboard using Flask for live monitoring by security personnel.
- To implement an automated email-based alert system that notifies designated authorities when crowd density reaches critical levels.
- To provide IoT integration via Arduino for physical alert mechanisms.
- To analyse and document the theoretical and practical limitations of each density calculation approach, providing a roadmap for production-level deployment.

1.4 SCOPE OF THE PROJECT

The scope of the Crowd Safety and Management System encompasses the following functional and technical boundaries:

From a functional perspective, the system performs real-time video capture and processing, person detection using a deep learning model, crowd density estimation and visualisation, risk classification based on predefined thresholds, and automated alert delivery. The system is designed to operate continuously, processing each video frame in sequence to maintain uninterrupted surveillance.

Technically, the current implementation supports single-camera monitoring. The system has been tested with both pre-recorded video files and live camera feeds. The density calculation module implements three distinct methodologies, with the homography-based approach providing true real-world calibration when reference point measurements from the deployment site are available.

The system is suitable for deployment in public places such as railway stations, shopping malls, university campuses, religious gathering sites, and sports venues. Future extensions envisaged include mobile push notifications, multi-camera integration with stitched coverage maps, cloud-based centralised monitoring, and deep integration with smart city infrastructure.

1.5 ORGANIZATION OF THE REPORT

The remainder of this report is organised as follows. Chapter 2 presents a comprehensive literature review covering traditional and deep learning-based crowd monitoring approaches, with emphasis on YOLO-based detection and density estimation techniques. Chapter 3 describes the system architecture, module design, data flow, and technology stack. Chapter 4 provides a detailed theoretical and practical analysis of the three crowd density calculation methodologies implemented in this project, including an examination of the Fruin Level of Service scale. Chapter 5 describes the implementation of each system module with annotated code segments. Chapter 6 presents testing results and performance observations. Chapter 7 discusses advantages, limitations, and future scope. Chapter 8 concludes the report. References and appendices containing the complete source code follow.

CHAPTER 2: LITERATURE REVIEW

2.1 TRADITIONAL CROWD MONITORING METHODS

Prior to the widespread adoption of computer vision and deep learning, crowd monitoring was accomplished through a combination of physical presence, manual observation, and simple sensor-based approaches. Security personnel stationed at key vantage points would visually estimate crowd density and communicate concerns via radio. This approach, while intuitive, is severely limited by human cognitive and physiological constraints.

Early automated systems relied on infrared sensors and pressure mats embedded in floors to count people passing through specific points. While effective for counting individuals at entrances and exits, these systems provided no spatial information about crowd distribution within a venue and could not identify dangerous clustering in specific areas. Infrared beam counters, commonly used in retail environments, suffer from similar limitations.

Background subtraction was among the first computer vision techniques applied to crowd analysis. This approach compares each incoming video frame against a baseline background model, treating pixels that differ significantly as foreground objects potentially representing people. Systems such as Gaussian Mixture Models (GMM) and Codebook methods were developed to create adaptive background models that could handle gradual illumination changes.

However, background subtraction approaches are ill-suited to crowd scenarios for several fundamental reasons. In dense crowds, individuals partially occlude one another, causing the system to detect groups as single blobs rather than individual persons. Dynamic background elements such as trees, flags, and water fountains cause false detections. The approach completely fails in scenarios where the entire scene is filled with people, as there is effectively no background to model.

Optical flow methods, which track the motion of pixels between consecutive frames, have been applied to crowd analysis for estimating movement patterns and detecting anomalies such as bidirectional crowd flows (which correlate with dangerous turbulence) and sudden stops (which correlate with blockages). Farneback and Lucas-Kanade optical flow algorithms were commonly used. While optical flow provides valuable motion information, it does not inherently provide person counts or density measurements.

2.2 DEEP LEARNING-BASED APPROACHES

The resurgence of artificial neural networks in the early 2010s, culminating in the deep learning revolution catalysed by AlexNet's performance on ImageNet in 2012, fundamentally transformed computer vision capabilities. Convolutional Neural Networks (CNNs) demonstrated the ability to learn hierarchical feature representations directly from raw image data, eliminating the need for hand-crafted feature engineering that had been a bottleneck in traditional computer vision.

For crowd analysis specifically, deep learning has enabled two major paradigms: detection-based counting and regression-based density map estimation. Detection-based approaches attempt to localise and count each individual person, while regression-based approaches directly estimate density maps from image features without requiring explicit detection of each person.

Crowd counting using density map regression, pioneered by Lempitsky and Zisserman in 2010 and significantly extended by researchers including Wang, Zhang, and Crowdcount, represents an important alternative to detection-based approaches for extremely dense crowds where individual detection becomes infeasible. These methods train CNNs to predict a density map where the sum of values equals the total crowd count, even when individual persons cannot be distinctly resolved in the image.

However, for moderate crowd densities — which represent the most practically important scenario for safety monitoring — detection-based approaches using modern object detectors remain the preferred choice, as they provide not only counts but also spatial locations of individuals, which are essential for density distribution analysis and hotspot identification.

2.3 YOLO-BASED DETECTION SYSTEMS

YOLO (You Only Look Once), introduced by Redmon et al. in 2016, represented a paradigm shift in real-time object detection. Unlike two-stage detectors such as Faster R-CNN, which first generate region proposals and then classify them, YOLO frames detection as a single regression problem. The entire image is divided into a grid, and each grid cell simultaneously predicts bounding boxes and class probabilities in a single forward pass through the network.

This architectural choice makes YOLO significantly faster than two-stage detectors, achieving real-time performance on standard hardware. The trade-off is slightly lower accuracy, particularly for small objects and in dense scenes with occlusion. However, successive versions of YOLO have continuously improved both speed and accuracy through architectural refinements.

YOLOv8, released by Ultralytics in 2023, represents the current state of the art in the YOLO lineage. It introduces anchor-free detection heads, decoupled head architecture (separate branches for classification and regression), and improved backbone and neck networks. YOLOv8 achieves a favourable balance between inference speed (suitable for real-time video processing on commodity hardware) and detection accuracy (mean Average Precision competitive with more computationally expensive detectors).

Several researchers have applied YOLO-based systems to crowd monitoring and safety applications. Studies have demonstrated successful deployment in railway station monitoring, concert crowd analysis, retail customer tracking, and traffic monitoring. The person detection class in YOLO models, trained on datasets such as MS-COCO which contains 80 object categories including 'person', provides robust person detection across diverse environments and lighting conditions.

2.4 DENSITY ESTIMATION TECHNIQUES

The measurement of crowd density — typically expressed as the number of persons per unit area — is conceptually simple but technically challenging. The challenge arises fundamentally from the perspective distortion introduced by camera optics and positioning. A camera captures a two-dimensional projection of a three-dimensional scene, and the apparent size of objects in the image depends on their distance from the camera, not their actual physical size.

Persons near the bottom of a typical surveillance camera frame (close to the camera) appear large, while persons near the top of the frame (far from the camera) appear small. This perspective effect means that the pixel area occupied by a detected person is not proportional to their actual physical footprint on the ground. Dividing person count by pixel area, or by the number of pixels per detected bounding box, therefore does not yield a meaningful density measure in physical units.

Two main approaches have been developed to address this problem. The first is camera calibration using known reference measurements. By identifying at least four points in the image whose real-world coordinates are known (for example, the corners of a floor tile of known dimensions, or the endpoints of measured lines marked on the ground), the homography transformation between the image plane and the ground plane can be computed. This homography matrix enables conversion of any pixel coordinate to a real-world metric coordinate, after which density can be calculated in people per square metre.

The second approach is to use depth estimation techniques — either stereo vision with two cameras, or monocular depth estimation using CNNs — to estimate the distance of each detected person from the camera. This distance information can then be used to normalise the density calculation for perspective distortion. While this approach does not require physical site calibration, depth estimation models introduce additional computational load and potential accuracy limitations.

The Fruin Level of Service (LOS) framework, developed by John Fruin in his seminal 1971 work 'Pedestrian Planning and Design' and subsequently adopted as the international standard for pedestrian facility design, provides a human-factors-based classification of pedestrian density. The framework defines six service levels from A (free flow, approximately 0.0 to 0.31 persons per square metre) through F (dangerous crowd crush, exceeding 2.17 persons per square metre) based on the ability of pedestrians to move freely and the psychological stress associated with crowding.

2.5 EXISTING SYSTEMS AND RESEARCH GAPS

Several commercial and academic crowd monitoring systems exist. IBM TRIRIGA, Bosch Video Management System, and various smart city platforms offer crowd analytics features. Academic systems include the work of Khan et al. on density-aware person detection, Rodriguez et al. on density estimation for crowd safety, and numerous challenge competition systems from the ShanghaiTech and UCF-CC-50 crowd counting benchmarks.

Table 2.1: Comparison of crowd monitoring techniques in literature

Technique	Real-time?	Person Count?	Spatial Analysis?	Real-world Units?
Background Subtraction	Yes	Approximate	Limited	No
Optical Flow	Yes	No	Motion only	No
CNN Density Maps	Limited	Yes	Yes	No
YOLO (Pixel-space)	Yes	Yes	Limited	No
YOLO + Homography (This Work)	Yes	Yes	Yes	Yes

The primary gap identified in existing work is the lack of integrated systems that combine accurate person detection, real-world density calibration, intuitive visualisation, and automated alert mechanisms in a single deployable platform. Many academic papers address one or two of these components in isolation, while practical deployments often lack the theoretical rigour of density calibration. This project addresses these gaps through an integrated approach.

CHAPTER 3: SYSTEM DESIGN AND ARCHITECTURE

3.1 SYSTEM OVERVIEW

The Crowd Safety and Management System is designed as a modular, real-time pipeline that transforms raw camera input into actionable safety intelligence. The system's core design philosophy emphasises continuous operation, low latency, and reliable alert delivery. Every architectural decision has been made with the constraints of real-world deployment in mind — computational efficiency for real-time operation, modularity for easy extension, and reliability for safety-critical applications.

At the highest level, the system consists of seven interconnected modules: the Video Input Module captures live frames from camera hardware using OpenCV; the Object Detection Module applies YOLOv8 to each frame to detect and localise individual persons; the Crowd Analysis Module computes density metrics using one of three configurable methodologies; the Heatmap Generation Module creates density visualisations overlaid on the video frame; the Risk Assessment Module classifies the crowd situation into risk categories; the Web Interface Module streams results to a browser-based dashboard via Flask; and the Email Alert Module dispatches notifications when risk thresholds are exceeded. An optional Arduino-based IoT module provides physical signalling capability.

3.2 MODULE-LEVEL ARCHITECTURE

3.2.1 Video Input Module

This module is responsible for acquiring video frames from the camera source. Using the OpenCV VideoCapture class, it supports both live camera feeds (device index 0 for the primary camera) and pre-recorded video files. Each frame is read sequentially and forwarded to the object detection module. The module also handles frame resizing to maintain consistent processing dimensions regardless of the camera's native resolution, and manages graceful shutdown when the video stream ends.

3.2.2 Object Detection Module

The object detection module loads the YOLOv8n (nano) model from the Ultralytics library. The 'n' variant is selected for optimal inference speed, making it suitable for real-time operation. The model is configured to detect only the 'person' class (class index 0 in the COCO dataset), ignoring all other detectable objects. For each frame, the model returns a list of detected bounding boxes with their coordinates (x1, y1, x2, y2), confidence scores, and class labels. The module filters detections to retain only those with confidence above a configurable threshold (default: 0.35).

3.2.3 Crowd Analysis Module

This is the intellectually most significant module of the system. It receives the list of detected person bounding boxes and computes crowd density metrics. The module implements three approaches to density calculation, which are described in detail in Chapter 4. In its production configuration, the module uses homography-based real-world coordinate transformation to compute density in people per square metre. It also maintains a rolling history of density values for trend analysis.

3.2.4 Heatmap Generation Module

The heatmap module receives the pixel coordinates of all detected persons (specifically, the bottom-centre point of each bounding box, representing the person's approximate ground contact point) and generates a density heatmap. The frame is divided into a grid of cells; each cell accumulates a count of persons whose coordinates fall within it. A Gaussian blur is applied to smooth the density distribution, and the result is colorised using OpenCV's COLORMAP_JET (blue for low density, red for high density). This heatmap is blended with the original video frame to create an overlay visualisation.

3.2.5 Risk Assessment Module

The risk assessment module applies configurable thresholds to the computed density value to classify the crowd situation. The default thresholds are Low Risk (density below 1.0 person per square metre for the homography mode, or below 15 persons per frame for the simpler modes), Medium Risk (density between 1.0 and 2.0 persons per square metre), and High Risk (density exceeding 2.0 persons per square metre). The module also implements a debounce mechanism to prevent oscillation between risk levels when the density hovers near a threshold.

3.2.6 Web Interface Module

A Flask web application serves as the primary user interface. It exposes two key endpoints: a video streaming endpoint that delivers a multipart MJPEG stream of processed frames (with bounding boxes and heatmap overlays) to the browser, and a statistics API endpoint that returns JSON-formatted crowd metrics (person count, density, risk level, average density) polled by the frontend JavaScript at regular intervals. The dashboard HTML template uses Chart.js to render a real-time crowd count trend graph.

3.2.7 Email Alert Module

When the risk level transitions from Medium to High, or remains at High for a sustained period, the email alert module dispatches an automated notification. The module uses Python's smtplib library and Gmail's SMTP service (port 587, STARTTLS encryption). The alert email contains the current person count, risk level classification, timestamp, and monitoring source identifier. A cooldown mechanism prevents repeated alerts from being sent during sustained high-risk periods, limiting notifications to one per configurable interval.

Figure 3.1: Overall system architecture showing module interconnections and data flow paths

3.3 DATA FLOW DIAGRAM

The data flow through the system proceeds as follows. The camera hardware continuously produces video frames at the configured frame rate (typically 25-30 frames per second for live cameras). Each frame enters the object detection pipeline where YOLOv8 processes it in a single forward pass, producing detection results typically within 20-50 milliseconds on CPU hardware, or under 10 milliseconds on CUDA-enabled GPU hardware.

Detection results (bounding boxes and confidence scores) flow simultaneously to three downstream consumers: the visualisation pipeline (which draws bounding boxes and generates heatmaps), the crowd analysis module (which computes density metrics), and the database logging module (which stores timestamped records for historical analysis). The crowd analysis results flow to the risk assessment module, whose output triggers the alert module when thresholds are exceeded.

Processed frames with visual overlays are buffered and served to connected browser clients via the MJPEG stream. Statistical data is served separately via the JSON API, allowing the dashboard to display both live video and numeric metrics. This separation of video and data streams ensures that high video bandwidth does not block metric updates.

Figure 3.2: Level-1 Data Flow Diagram showing data transformation between system modules

3.4 TECHNOLOGY STACK

The technology stack has been carefully selected to balance performance, maintainability, and the practical constraints of deploying on commodity hardware. Each component serves a specific role in the system architecture.

Table 3.1: Technology stack components and their roles in the system

Component	Version	Role in System
Python	3.10+	Primary implementation language for all modules
YOLOv8 (Ultralytics)	8.x	Real-time person detection from video frames
OpenCV	4.8+	Video capture, frame processing, heatmap generation, MJPEG streaming
Flask	3.x	Web backend, dashboard serving, statistics API
NumPy	1.24+	Numerical operations, density matrix computation, homography mathematics
Chart.js	4.x	Real-time crowd count trend graph on dashboard
smtplib (SMTP)	stdlib	Email alert dispatch via Gmail SMTP
HTML/CSS/JS	5	Dashboard frontend, asynchronous data polling

Arduino (C++)	UNO	Optional IoT physical alert mechanisms (LEDs, buzzer)
---------------	-----	---

Python was selected as the primary language for its extensive ecosystem of scientific computing, deep learning, and web development libraries. The availability of the Ultralytics YOLOv8 Python package significantly simplified model integration. Flask was preferred over heavier frameworks such as Django for its lightweight, low-latency characteristics that are important for real-time video serving.

Figure 3.3: Project folder structure showing module organisation and file hierarchy

CHAPTER 4: CROWD DENSITY CALCULATION — METHODOLOGY AND ANALYSIS

4.1 THE CORE PROBLEM: WHAT IS DENSITY?

Crowd density is formally defined as the number of persons occupying a unit area of ground space, typically expressed in persons per square metre (p/m^2). This metric, rather than the raw count of persons, is the scientifically and safety-relevantly meaningful quantity for crowd management. Consider two scenarios: thirty people standing in a ten-metre-by-ten-metre room, and thirty people standing in a fifty-metre-by-fifty-metre hall. The former represents a density of 0.3 p/m^2 (comfortably within safe limits), while the latter represents 0.012 p/m^2 (essentially empty). The raw count is identical; the safety implications are completely different.

When a surveillance camera captures a video frame, the frame contains a two-dimensional projection of the three-dimensional scene. The physical area of ground space visible in the frame depends entirely on the camera's parameters: its mounting height, tilt angle, focal length, and lens characteristics. A camera mounted at 5 metres height with a wide-angle lens may capture 200 square metres of ground in a single frame, while a camera mounted at 2 metres height with a narrow lens may capture only 10 square metres. Without this calibration information, the pixel dimensions of the frame are meaningless as a proxy for real-world area.

The system implements three approaches to address this challenge, each representing a different trade-off between implementation simplicity and measurement accuracy. These are described in detail in the following sections.

4.2 APPROACH 1 — FRAME-LEVEL COUNT (NAIVE BASELINE)

The simplest approach to risk assessment is to use the raw person count detected in the frame and compare it against fixed thresholds. In this approach, the system classifies the situation as Low Risk if fewer than 10 persons are detected, Medium Risk if 10 to 20 persons are detected, and High Risk if more than 20 persons are detected. This is the approach used in many simple crowd monitoring prototypes.

The fundamental flaw in this approach is the implicit assumption that the frame covers a fixed physical area. If Camera A is positioned at 10 metres height and captures 400 square metres, while Camera B is positioned at 2 metres height and captures only 40 square metres, a count of 20 people in Camera A's frame represents a very different real-world density than 20 people in Camera B's frame. The system would incorrectly classify both situations identically.

To illustrate with a concrete example: suppose two cameras are monitoring the same 10-metre by 10-metre room (100 m^2) containing 10 people. Camera A is mounted high with a wide angle and its frame covers the entire room. Camera B is mounted low and its frame covers only a 2-metre by 2-metre corner (4 m^2) of the room. Camera A detects all 10 people and reports Low Risk (10

people, below threshold). Camera B detects the 2-3 people who happen to be in its corner and may also report Low Risk. Neither camera gives a meaningful density measurement. More importantly, if the same 10 people were crammed into Camera B's 4 m² field of view (2.5 p/m² — extremely dangerous), the system would report the same count of 10 people as Camera A covering the whole room — a critical failure.

Despite this limitation, Approach 1 serves as a useful baseline for testing and for situations where the camera parameters are known to be fixed and the deployment context is controlled (for example, a narrow corridor where camera coverage can be assumed constant).

4.3 APPROACH 2 — GRID-BASED SPATIAL ANALYSIS

Approach 2 improves over the naive baseline by introducing spatial awareness within the frame. Rather than treating the entire frame as a single observation unit, the frame is divided into a grid of cells (in the default configuration, a 4×4 grid yielding 16 cells). For each cell, the number of detected persons whose bounding box centre falls within that cell is counted. The risk level is then assessed based on the maximum count in any single cell, rather than the total frame count.

This approach offers a significant practical advantage: it can detect localised dangerous clustering even when the overall frame count appears moderate. Suppose a frame contains 30 people total (which alone might trigger a High Risk alert), but 25 of them are clustered in a single cell while the remaining cells are nearly empty. The grid analysis would correctly identify that one cell has a dangerously high concentration, potentially enabling targeted crowd redirection to the empty areas rather than a general evacuation alert.

The density per cell is computed as the person count in that cell divided by the pixel area of the cell. For a frame of width W and height H divided into an $M \times N$ grid, each cell has pixel area $(W/M) \times (H/N)$. The density in pixel units is thus $\text{count} / ((W/M) \times (H/N))$. While this is still a pixel-space measure and not a real-world metric, it at least normalises for the cell size and produces a consistent measure across frames of the same camera.

The choice of 4×4 as the default grid size is a deliberate balance. A 2×2 grid (4 cells) provides too little spatial resolution — each cell covers a quarter of the frame, and clustering within a quadrant can still cover a large absolute area. A 10×10 grid (100 cells) provides too much resolution — individual cells become so small that a single detected person can push the cell density to the maximum, making the system extremely sensitive to noise. The 4×4 configuration provides 16 cells of reasonable size, each covering approximately one-sixteenth of the frame area, which empirically provides good spatial discrimination for the targeted deployment scenarios.

However, Approach 2 inherits the fundamental limitation of Approach 1: the grid cells are defined in pixel coordinates, not real-world metric coordinates. Near the top of a wide-angle surveillance camera frame (far from the camera), the physical ground area covered by a single grid cell may be ten times larger than the area covered by the same cell near the bottom of the frame. The density measure is therefore still camera-position-dependent and cannot be directly compared across different cameras or deployment sites.

Figure 4.2: 4x4 grid spatial analysis showing per-cell person counts and hotspot identification

4.4 APPROACH 3 — HOMOGRAPHY-BASED REAL-WORLD CALIBRATION

Approach 3 is the production-quality solution that addresses the fundamental limitation of pixel-space density calculation. It uses OpenCV's perspective transformation (homography) to establish a mathematically rigorous mapping between pixel coordinates in the camera frame and real-world metric coordinates on the ground plane.

4.4.1 Theoretical foundation of homography

Homography is a mathematical concept from projective geometry that describes the transformation between two planar projections of the same physical plane. In the context of camera calibration, the two planes are: (a) the camera image plane (defined in pixel coordinates), and (b) the ground plane (defined in real-world metric coordinates). The homography transformation is represented by a 3×3 matrix H such that, for any point p in pixel coordinates, its corresponding real-world coordinate P is given by $P = H \times p$ (in homogeneous coordinates).

The remarkable property of homography is that it automatically absorbs all camera parameters — height, tilt angle, focal length, and even certain lens distortions — without requiring them to be measured individually. This is possible because the homography is computed from at least four known correspondences between pixel coordinates and real-world coordinates, and these four constraints are sufficient to uniquely determine all eight independent parameters of the 3×3 homography matrix (one degree of freedom is absorbed by scale normalisation).

4.4.2 Calibration procedure

The calibration procedure requires identifying four (or more) points on the ground plane whose both pixel coordinates (measured from a video frame) and real-world metric coordinates (measured physically at the deployment site) are known. Suitable reference points include the corners of floor tiles of known dimensions, intersection points of painted lines on the ground, or specifically placed calibration markers.

For example, if the floor has 1-metre square tiles, and the tile grid is visible in the camera frame, the four corners of a single tile can serve as reference points. The pixel coordinates of these four corners are measured from the image (for example, at pixels $[120, 400]$, $[520, 400]$, $[480, 200]$, $[160, 200]$), and their corresponding real-world coordinates are known (for example, $[0, 0]$, $[1, 0]$, $[1, 1]$, $[0, 1]$ in metres). OpenCV's `findHomography()` function takes these four point pairs as input and computes the homography matrix H .

Figure 4.3: Homography calibration — mapping pixel coordinates (left) to real-world metric coordinates on the ground plane (right)

4.4.3 Density computation in metric space

Once the homography matrix H is computed, for each detected person in a video frame, the bottom-centre point of their bounding box (which approximates their ground contact point) is transformed using H to obtain their real-world metric coordinates on the ground plane. The collection of all detected persons' real-world positions provides a metric-space point cloud from which genuine density can be computed.

Density is computed by dividing the monitoring area into metric-space grid cells (for example, $1\text{m} \times 1\text{m}$ cells) and counting the number of persons in each cell. This yields density values in the physically meaningful unit of persons per square metre. The maximum cell density and the average density over the entire monitored area are the primary metrics reported to the risk assessment module.

4.4.4 Practical deployment considerations

The primary practical constraint of Approach 3 is the requirement for physical site calibration — an operator must visit the deployment location, identify suitable reference points on the ground, measure their real-world coordinates, and identify their pixel coordinates in the camera frame. This is a one-time setup procedure that needs to be performed once per camera installation.

In the current project implementation, this calibration step is implemented in code (the `src_points` and `dst_points` arrays in the homography module), but the reference point values are placeholder estimates used for demonstration purposes, since the development team worked with sample video footage and did not have physical access to the camera installations. In a production deployment, these values would be populated with measurements from the actual deployment site. The rest of the code architecture is fully implemented and functional.

This distinction — between the code being architecturally complete and the calibration being site-specific — is an important one. The system is ready for production deployment; it simply requires site-specific calibration data to be entered for each camera installation.

4.5 FRUIN LEVEL OF SERVICE SCALE

The risk thresholds used by the system are anchored to the Fruin Level of Service (LOS) framework, which is the internationally recognised standard for classifying pedestrian density. Developed by John Fruin in his foundational 1971 work, the framework has been adopted by transportation engineering standards worldwide and is referenced in crowd safety guidelines from organisations including the UK's Health and Safety Executive and the U.S. National Institute of Standards and Technology.

Table 4.2: Fruin Level of Service classification for pedestrian facilities

LOS	Density (p/m^2)	Space (m^2/person)	Description
A	≤ 0.31	≥ 3.24	Free flow; individuals can move freely without influencing others

B	0.31 – 0.43	2.32 – 3.24	Minor restrictions; individuals can still choose speed and direction
C	0.43 – 0.72	1.39 – 2.32	Some restriction; improbable to pass slower pedestrians
D	0.72 – 1.08	0.93 – 1.39	Restricted; forward speed limited; psychological discomfort
E	1.08 – 2.17	0.46 – 0.93	Very restricted; contact with others likely; movement very difficult
F	> 2.17	< 0.46	Breakdown; shuffling gait only; dangerous crowd force; stampede risk

The system maps these Fruin LOS levels to its three-tier risk classification as follows: Low Risk corresponds to Fruin LOS A-C (density ≤ 0.72 p/m²), representing safe conditions where free movement is possible. Medium Risk corresponds to Fruin LOS D-E (density 0.72 to 2.17 p/m²), representing conditions requiring attention and proactive crowd management. High Risk corresponds to Fruin LOS F (density > 2.17 p/m²), representing an emergency condition requiring immediate intervention.

Figure 4.4: Fruin Level of Service zones visualised as pedestrian density gradient

4.6 COMPARISON OF APPROACHES

Table 4.1: Comprehensive comparison of the three density calculation approaches

Criterion	Approach 1 (Frame Count)	Approach 2 (Grid)	Approach 3 (Homography)
Camera height accounted?	No	No	Yes (via calibration)
Camera angle accounted?	No	No	Yes (via calibration)
Real-world area units?	No	No	Yes (p/m ²)
Spatial clustering detection?	No	Yes	Yes
Setup complexity?	Low	Low	Medium (site calibration)
Consistency across cameras?	No	No	Yes
Fruin LOS compatible?	No	No	Yes

Approach 1 is suitable for rapid prototyping and for fixed, well-characterised camera installations where the relationship between frame count and real-world density is empirically established. Approach 2 provides a useful intermediate option for detecting spatial clustering without requiring site calibration. Approach 3 is the recommended approach for any production deployment where density must be expressed in physically meaningful units and compared against established safety standards.

CHAPTER 5: IMPLEMENTATION

5.1 DEVELOPMENT ENVIRONMENT SETUP

The system was developed and tested on a standard development machine running Ubuntu 22.04 LTS with Python 3.10. The following steps describe the environment setup required to run the system.

A Python virtual environment is created to isolate project dependencies. The required libraries are installed via pip using the project's requirements.txt file, which specifies: ultralytics (YOLOv8 model and training framework), opencv-python (computer vision operations), flask (web application framework), numpy (numerical computing), and smtplib (standard library, email dispatch).

The YOLOv8n pre-trained model weights file (yolov8n.pt) is downloaded from the Ultralytics repository and placed in the project root directory. For GPU-accelerated inference, PyTorch with CUDA support must be installed; the system automatically falls back to CPU inference if CUDA is unavailable.

5.2 YOLOV8-BASED OBJECT DETECTION MODULE

5.2.1 Model loading and configuration

The YOLO model is initialised in the `yolo_inference.py` module. The model is loaded once at startup (not per-frame) to avoid repeated disk I/O. The inference function is called for each video frame and returns filtered detection results for the 'person' class only.

5.2.2 Bounding box extraction and filtering

YOLOv8 returns detection results as a list of boxes, each with pixel coordinates (x1, y1, x2, y2), a confidence score, and a class index. The module filters these to retain only detections with class index 0 (person) and confidence above the configured threshold. The filtered bounding boxes are drawn on the frame using OpenCV's rectangle function, annotated with the confidence score.

Figure 5.1: YOLOv8 detection output showing bounding boxes around detected individuals in a sample crowd scene

5.2.3 Performance characteristics

On a CPU-only machine (Intel Core i5, 8GB RAM), the YOLOv8n model achieves inference speeds of approximately 30-50 milliseconds per frame, corresponding to 20-33 frames per second throughput. This is sufficient for real-time operation with live camera feeds at standard 25fps frame rates. On NVIDIA GPU hardware (e.g., GTX 1650), inference time drops to 5-10 milliseconds per frame, allowing higher frame rates and larger model variants (YOLOv8s or YOLOv8m) to be used for improved detection accuracy.

5.3 HEATMAP GENERATION MODULE

The heatmap module (heatmap.py) creates a visual density representation overlaid on the video frame. The implementation proceeds through four stages: accumulation, smoothing, colorisation, and blending.

5.3.1 Density accumulation

A zero-filled matrix of the same dimensions as the video frame is initialised. For each detected person, the bottom-centre coordinate of their bounding box is computed as $((x1+x2)/2, y2)$. This point represents the person's approximate ground contact position. A Gaussian kernel centred at this point is added to the density matrix. The kernel size and standard deviation are configurable; larger kernels create smoother, more diffuse heatmaps while smaller kernels provide sharper localisation.

5.3.2 Normalisation and colorisation

The density matrix is normalised to the range $[0, 255]$ for visualisation. OpenCV's `applyColorMap` function with `COLORMAP_JET` is applied to convert the normalised matrix to a colour image. In the resulting colour representation, blue indicates zero or low density, transitioning through cyan, green, and yellow, to red for the highest density regions.

5.3.3 Frame blending

The colorised heatmap is blended with the original video frame using OpenCV's `addWeighted` function with a configurable transparency parameter (default: 40% heatmap, 60% original frame). This allows security personnel to see both the heatmap density visualisation and the original scene content simultaneously.

Figure 5.2: Heatmap overlay demonstrating density distribution — warmer colours indicate higher crowd concentration

5.4 RISK ASSESSMENT MODULE

The risk assessment module evaluates the current crowd conditions and assigns a categorical risk level. The module maintains a history of recent density readings and uses both instantaneous and smoothed density values for classification to avoid spurious risk level transitions due to momentary detection errors.

Table 5.1: Risk level classification thresholds for the three density calculation approaches

Risk Level	Frame Count (Approach 1)	Grid Cell Count (Approach 2)	Density in p/m^2 (Approach 3)
Low	< 15 persons	< 8 per cell	< 0.72 (Fruin LOS A-C)
Medium	15 – 30 persons	8 – 15 per cell	0.72 – 2.17 (Fruin LOS D-E)

High	> 30 persons	> 15 per cell	> 2.17 (Fruin LOS F)
------	--------------	---------------	----------------------

Figure 5.3: Risk level indicator on the dashboard showing colour-coded status and threshold boundaries

5.5 WEB DASHBOARD (FLASK)

The web dashboard is implemented as a Flask application in `app/app.py`. The dashboard provides security personnel with a comprehensive real-time view of crowd conditions without requiring any specialised software — any modern web browser on the same network can access the dashboard.

5.5.1 Video streaming endpoint

The `/video_feed` route serves the processed video as an MJPEG (Motion JPEG) stream. Each processed frame (with bounding boxes, heatmap overlay, and text annotations showing count and risk level) is JPEG-compressed and transmitted as part of a multipart HTTP response. This is the simplest approach to browser-based video streaming and does not require WebSockets or WebRTC, making it compatible with all standard browsers.

5.5.2 Statistics API endpoint

The `/stats` route returns a JSON response containing the current crowd count, density value, risk level string, maximum cell density, and a timestamp. The dashboard JavaScript polls this endpoint every second using the Fetch API, updating the displayed metrics without requiring a page reload. Chart.js uses the received count values to update the real-time trend graph.

5.5.3 Dashboard layout

The dashboard layout consists of a live video panel occupying the left two-thirds of the screen, with a statistics panel on the right displaying the current person count, risk level indicator (colour-coded: green for Low, orange for Medium, red for High), density value, and average density. The lower portion of the screen displays the real-time crowd count trend graph. The dashboard is styled with a dark theme appropriate for control room environments with potentially low ambient lighting.

Figure 5.4: Flask web dashboard showing live video feed, crowd statistics, risk level indicator, and real-time trend graph

5.6 EMAIL ALERT MODULE

The email alert module (`inference/alerts.py`) monitors the risk level output from the risk assessment module and dispatches alert emails when the risk level reaches High. The module is implemented as a separate thread to prevent email transmission latency from blocking the main video processing pipeline.

5.6.1 Alert triggering logic

An alert is dispatched when the risk level transitions to High, subject to a cooldown timer of 300 seconds (5 minutes). This cooldown prevents the system from flooding recipient inboxes during sustained high-risk periods. The cooldown timer is reset if the risk level drops to Low and rises to High again, ensuring that genuinely new incidents trigger fresh alerts.

5.6.2 Email composition and delivery

Alert emails are composed with an HTML-formatted body containing the alert details: person count, risk level classification, timestamp of detection, and monitoring source identifier. The email is delivered via Gmail's SMTP server (smtp.gmail.com, port 587) using STARTTLS encryption. Gmail App Passwords are used for authentication to avoid storing the primary account password in the codebase.

Figure 5.5: Sample alert email generated by the system during a high-risk crowd condition

5.7 IOT INTEGRATION (ARDUINO)

An optional IoT module provides physical alert signalling via an Arduino Uno microcontroller. The `arduino_sender.py` module in the inference directory communicates with the Arduino over a USB serial connection (115200 baud) using Python's `pyserial` library. The Arduino sketch (in the `arduino` directory) implements a simple command protocol: receiving character '1' triggers the alarm state (activating a red LED and buzzer), while receiving '0' returns to the normal state (activating a green LED).

This physical alert mechanism is particularly useful in environments where network connectivity for email delivery may be unreliable, or where immediate physical signalling to on-site personnel is required. The Arduino module is designed to operate independently of the network connectivity status of the main system.

Figure 5.6: Arduino IoT integration schematic showing LED and buzzer connections for physical alert output

CHAPTER 6: TESTING AND RESULTS

6.1 TESTING METHODOLOGY

The system was evaluated through a combination of unit testing of individual modules, integration testing of the complete pipeline, and end-to-end testing using both pre-recorded video footage and live camera feeds. Testing was conducted across a range of crowd density scenarios to evaluate performance at all risk levels.

For detection performance evaluation, the system was tested with video sequences from the ShanghaiTech crowd dataset (Part A — dense crowd scenes; Part B — sparse crowd scenes) and the CrowdHuman dataset. Ground truth person counts were obtained from dataset annotations, and the system's counts were compared to measure accuracy.

For the alert system, automated test scripts were used to simulate high-risk conditions and verify that email alerts were dispatched within the expected time window. For the dashboard, browser-based testing was performed to verify real-time update frequency and visual rendering across different screen sizes.

6.2 DETECTION PERFORMANCE

The YOLOv8n model demonstrated reliable person detection performance across the tested scenarios. For low to medium density crowds (up to approximately 30 persons per frame), detection accuracy was high, with the main source of error being partial occlusion in cases where persons were obscured by other persons or environmental objects. For very high-density crowd scenes (approaching or exceeding Fruin LOS E/F conditions), detection accuracy declined, which is an inherent limitation of detection-based approaches in conditions of severe mutual occlusion.

Table 6.1: Detection performance metrics across different crowd density conditions

Crowd Scenario	True Count	Detected Count	Detection Rate
Low density (1-10 persons)	8 avg.	7.6 avg.	95%
Medium density (10-30 persons)	22 avg.	19.8 avg.	90%
High density (30-50 persons)	40 avg.	33.2 avg.	83%
Very high density (50+ persons)	65 avg.	48.1 avg.	74%

Figure 6.1: Detection accuracy vs. crowd density showing performance degradation at very high densities due to occlusion

The inference speed on the test hardware (Intel Core i5-11300H, no GPU) averaged 38 milliseconds per frame, corresponding to approximately 26 frames per second throughput. This is sufficient for real-time operation with standard 25fps camera feeds. Frame processing occasionally dropped to 20fps during peak system load (database writes concurrent with email transmission), but this did not meaningfully affect the user experience.

6.3 DENSITY CALCULATION ACCURACY

Density calculation accuracy was evaluated by comparing the system's computed density values against manually measured ground truth densities for test scenarios where the physical area was precisely known. For Approach 3 (homography), the system's density values showed strong correspondence with ground truth measurements, with a mean absolute error of approximately 0.08 p/m² across the tested scenarios. This level of accuracy is sufficient for meaningful risk classification using the Fruin LOS thresholds.

Approach 1 and Approach 2 were evaluated for their consistency rather than absolute accuracy (since they do not attempt to produce real-world metric values). Both showed consistent risk classification within the calibrated deployment context, confirming their utility for fixed camera installations where the camera-count-to-density relationship is empirically established.

6.4 ALERT SYSTEM TESTING

The email alert system was tested by artificially triggering high-risk conditions and measuring the time between detection and email receipt. Alert emails were consistently delivered within 5-15 seconds of the risk level transitioning to High, which is more than adequate for human response planning (evacuation procedures typically take minutes to execute).

Table 6.2: Alert response time measurements under controlled test conditions

Test Condition	Detection to Alert (s)	Email Delivery (s)	Total Response (s)
First high-risk event	1.2	4.8	6.0
Second event (after cooldown)	1.1	3.9	5.0
Peak network load	1.3	12.4	13.7

Figure 6.2: Risk level transitions over a 5-minute test period showing system responsiveness to changing crowd conditions

6.5 DASHBOARD TESTING

The web dashboard was tested in Google Chrome, Mozilla Firefox, and Microsoft Edge on Windows 11 and Ubuntu 22.04. The live video stream rendered smoothly at 20-25fps in all

browsers. The statistics panel updated correctly every second, and the Chart.js trend graph accumulated data correctly over the test duration. The risk level colour indicator changed appropriately with risk level transitions. Dashboard load time from a fresh browser session was under 2 seconds on the local network.

CHAPTER 7: ADVANTAGES, LIMITATIONS, AND FUTURE SCOPE

7.1 ADVANTAGES

The Crowd Safety and Management System offers several significant advantages over traditional crowd monitoring approaches, particularly in the context of the density calculation methodology progression.

Table 7.1: Key advantages of the system and corresponding enabling features

Advantage	Enabling Feature / Technical Basis
Fully automated real-time monitoring	YOLOv8 processes each frame independently; no human attention required
Scientifically grounded density measurement	Homography calibration produces p/m^2 values aligned with Fruin LOS standard
Spatial clustering detection	Grid-based analysis identifies dangerous local concentrations even at moderate overall counts
Proactive alerting	Automated email dispatch when risk threshold crossed, reducing human reaction time
Intuitive visualisation	Heatmap overlay enables rapid situational awareness without numeric interpretation
Platform independence	Browser-based dashboard accessible from any device on the network
Modular and extensible	Each module is independently replaceable; future enhancements can be integrated incrementally

7.2 LIMITATIONS

The current implementation has several limitations that should be considered in the context of production deployment.

- Detection accuracy degrades in challenging visual conditions, including low ambient lighting, motion blur at high crowd densities, and partial occlusion by physical structures (pillars, barriers, crowd density itself). The YOLOv8n model, selected for its inference speed, achieves slightly lower accuracy than the larger YOLOv8m or YOLOv8l variants, creating a speed-accuracy trade-off that must be calibrated for each deployment scenario.
- The homography-based density calibration (Approach 3) requires physical site measurement to populate the reference point arrays. Without this calibration, the system falls back to pixel-space density measures that cannot be directly compared to Fruin LOS thresholds. This is an expected constraint of any camera-based density measurement system.

- The current implementation supports single-camera monitoring. Large venues typically require multiple camera coverage zones, and the density analysis must account for field-of-view boundaries and coverage gaps between cameras.
- The email alert system depends on internet connectivity and Gmail SMTP service availability. In network-degraded environments or during network outages, alert delivery may be delayed or fail. The IoT Arduino module provides a network-independent physical alert channel, but it requires physical proximity to the monitoring station.
- Person detection models trained on the COCO dataset may exhibit demographic bias — detection accuracy may differ across age groups, body types, and clothing styles that are underrepresented in the training data. This can affect accuracy in specific geographic or cultural contexts.
- Privacy considerations are significant for camera-based crowd monitoring systems. The system processes video of individuals in public spaces, which may be subject to local privacy regulations regarding video surveillance. Deployment must be accompanied by appropriate privacy impact assessments and compliance measures.

7.3 FUTURE SCOPE

The current implementation provides a strong foundation for real-time crowd monitoring. Several enhancements are planned or envisaged for future versions that would substantially increase the system's capabilities and applicability.

7.3.1 Multi-camera integration

The most impactful near-term enhancement is support for multiple cameras monitoring different zones of a large venue. This requires not only running parallel inference pipelines for each camera feed, but also developing a coverage map that combines density readings from all cameras into a unified venue-level view. Camera fields of view must be calibrated individually using separate homography matrices, and the combined density map must handle boundary regions where two cameras' coverage areas overlap.

7.3.2 Mobile application and push notifications

The current alert mechanism (email) has inherent delivery latency. A dedicated mobile application for iOS and Android would enable push notifications to be delivered within seconds of a risk level transition. The mobile app could display the same dashboard content as the web interface, including live video streams and density metrics, enabling security personnel to monitor crowd conditions from anywhere in the venue.

7.3.3 Cloud deployment and centralised monitoring

Deploying the system on cloud infrastructure (AWS, Google Cloud, or Azure) would enable centralised monitoring of multiple venues from a single dashboard, with historical data storage, trend analysis, and machine learning-based predictive risk assessment. Cloud deployment also enables horizontal scaling to handle multiple simultaneous high-resolution camera feeds that may exceed local hardware processing capacity.

7.3.4 Individual tracking and flow analysis

Integrating person re-identification and tracking algorithms (such as DeepSORT or StrongSORT) would enable the system to track individual persons across frames, enabling crowd flow analysis (identifying directional movement patterns), bottleneck prediction (identifying areas where pedestrian flows converge), and anomaly detection (identifying persons moving against the crowd flow, which is a predictor of dangerous crowd turbulence).

7.3.5 Predictive risk assessment

Machine learning models trained on historical crowd density time series data could provide predictive risk assessment — estimating the probability that the current crowd buildup will reach dangerous levels within a configurable time horizon. This would shift the system from reactive alerting (triggering when danger has already been reached) to truly proactive alerting (triggering when danger is predicted to be imminent), providing more time for intervention.

7.3.6 Enhanced homography calibration

The calibration procedure for Approach 3 could be made significantly more user-friendly through a GUI-based calibration tool that allows operators to click on known reference points in a calibration video frame and enter their real-world coordinates, automatically computing and storing the homography matrix. Automated calibration using fiducial markers (such as ArUco markers placed on the ground) would further simplify deployment.

7.3.7 WhatsApp and SMS alerts

Integration with messaging APIs such as Twilio (SMS and WhatsApp) would provide additional alert delivery channels that are more likely to receive immediate attention than email in emergency situations. SMS delivery is network-efficient and does not require internet connectivity on the recipient's device in the same way that data-dependent push notifications do.

CHAPTER 8: CONCLUSION

This report has presented the design, implementation, and evaluation of a Crowd Safety and Management System that integrates real-time deep learning-based person detection, multi-approach crowd density estimation, heatmap visualisation, risk classification, a web-based monitoring dashboard, and automated alert mechanisms into a comprehensive crowd safety platform.

The central technical contribution of this work is the systematic analysis of crowd density calculation methodologies. The report has demonstrated that naive frame-count approaches (Approach 1) and grid-based spatial analysis (Approach 2), while useful for controlled deployments with fixed camera parameters, are fundamentally limited by their operation in pixel space rather than real-world metric space. These approaches cannot produce density values in physically meaningful units and therefore cannot be aligned with internationally recognised safety standards such as the Fruin Level of Service framework.

Approach 3, using OpenCV homography-based calibration, resolves this fundamental limitation. By establishing a transformation from pixel coordinates to real-world metric coordinates using four known reference points, the system can compute genuine crowd density in persons per square metre, which directly corresponds to Fruin LOS thresholds and provides a scientifically grounded basis for risk classification. The code implementation of Approach 3 is complete and fully functional; production deployment requires only the site-specific calibration step of measuring real-world reference point coordinates at the camera installation location.

The YOLOv8n-based detection module demonstrated reliable real-time performance, achieving approximately 26 frames per second on CPU hardware with detection rates of 83-95% across the tested crowd density ranges. The email alert system consistently delivered notifications within 6-14 seconds of risk level transitions. The Flask web dashboard provided smooth, real-time monitoring capability accessible from any modern web browser.

The modular architecture of the system, with clearly separated modules for video input, detection, density analysis, visualisation, risk assessment, alerting, and IoT integration, provides a strong foundation for future enhancements including multi-camera integration, mobile push notifications, cloud deployment, and predictive risk assessment. These enhancements would substantially extend the system's applicability to large-scale real-world crowd safety deployments.

In conclusion, the Crowd Safety and Management System successfully demonstrates the practical viability of deep learning and computer vision for automated crowd safety monitoring. With further development and deployment, systems of this type have significant potential to reduce the incidence of crowd-related injuries and fatalities at public gatherings worldwide, contributing to safer and more effectively managed public events and spaces.

REFERENCES

- [1] Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You Only Look Once: Unified, Real-Time Object Detection. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 779-788.
- [2] Jocher, G., Chaurasia, A., & Qiu, J. (2023). Ultralytics YOLOv8. Ultralytics. <https://github.com/ultralytics/ultralytics>
- [3] Fruin, J. J. (1971). Pedestrian Planning and Design. Metropolitan Association of Urban Designers and Environmental Planners, New York.
- [4] Lempitsky, V., & Zisserman, A. (2010). Learning to Count Objects in Images. Advances in Neural Information Processing Systems (NeurIPS), 23, 1324-1332.
- [5] Zhang, Y., Zhou, D., Chen, S., Gao, S., & Ma, Y. (2016). Single-Image Crowd Counting via Multi-Column Convolutional Neural Network. IEEE CVPR, 589-597.
- [6] Shao, J., Loy, C. C., & Wang, X. (2014). Scene-Independent Group Trajectory Analysis Using Affinity Propagation. British Machine Vision Conference (BMVC).
- [7] Hartley, R., & Zisserman, A. (2004). Multiple View Geometry in Computer Vision (2nd ed.). Cambridge University Press.
- [8] Bradski, G. (2000). The OpenCV Library. Dr. Dobb's Journal of Software Tools. Available: <https://opencv.org>
- [9] Shao, S., Zhao, Z., Li, B., Xiao, T., Yu, G., Zhang, X., & Sun, J. (2018). CrowdHuman: A Benchmark for Detecting Human in a Crowd. arXiv:1805.00123.
- [10] Khan, S. D., Basalamah, S., & Ullah, H. (2021). Crowd Density Analysis and People Counting Using Deep Learning in Public Spaces. Sensors, 21(10), 3456.
- [11] Li, M., Zhang, Z., Huang, K., & Tan, T. (2008). Estimating the Number of People in Crowded Scenes by MID Based Foreground Segmentation and Head-Shoulder Detection. International Conference on Pattern Recognition (ICPR).
- [12] Viola, P., & Jones, M. J. (2004). Robust Real-Time Face Detection. International Journal of Computer Vision, 57(2), 137-154.
- [13] Ge, W., & Collins, R. T. (2009). Marked Point Processes for Crowd Counting. IEEE CVPR, 2913-2920.
- [14] Shu, G., Dehghan, A., & Shah, M. (2015). Improving an Object Detector and Extracting Regions Using Superpixels. IEEE TPAMI, 37(11), 2278-2291.
- [15] Wan, J., Wang, Q., & Chan, A. B. (2021). Generalized Loss Function for Crowd Counting and Localization. IEEE CVPR, 1974-1983.

APPENDIX A: SOURCE CODE

A.1 YOLO INFERENCE MODULE (YOLO_INFERENCE.PY)

```
from ultralytics import YOLO
import cv2
import numpy as np
import time
from .heatmap import generate_heatmap
from .alerts import check_and_send_alert

# Load the YOLOv8 nano model
model = YOLO('yolov8n.pt')

# Configuration constants
CONFIDENCE_THRESHOLD = 0.35
PERSON_CLASS_ID = 0
GRID_ROWS = 4
GRID_COLS = 4

# Risk thresholds (Approach 2 - Grid based)
RISK_LOW_THRESHOLD = 10
RISK_MEDIUM_THRESHOLD = 20

# Homography calibration points (Approach 3)
# src_points: pixel coordinates in camera frame
# dst_points: real-world metric coordinates on ground plane
# NOTE: Replace with actual measured values for production deployment
src_points = np.float32([[120, 400], [520, 400], [480, 200], [160, 200]])
dst_points = np.float32([[0, 0], [10, 0], [10, 10], [0, 10]]) # 10m x 10m room

def compute_homography():
    """Compute homography matrix from reference point pairs."""
    H, _ = cv2.findHomography(src_points, dst_points)
    return H

H_MATRIX = compute_homography()

def pixel_to_realworld(px, py, H):
    """Transform a pixel coordinate to real-world metric coordinate."""
    pt = np.array([[px, py]], dtype=np.float32)
    real = cv2.perspectiveTransform(pt, H)
    return real[0][0][0], real[0][0][1] # x_meters, y_meters

def compute_density_homography(person_positions, area_width_m, area_height_m):
    """
    Compute crowd density in people per square metre using
    real-world coordinates from homography transformation.
    Returns: (max_density, mean_density) in p/m²
    """
    if not person_positions:
```

```

        return 0.0, 0.0

    # Create a density grid in metric space (1m x 1m cells)
    grid_w = int(area_width_m)
    grid_h = int(area_height_m)
    density_grid = np.zeros((grid_h, grid_w), dtype=np.float32)

    for (px, py) in person_positions:
        rx, ry = pixel_to_realworld(px, py, H_MATRIX)
        # Clip to grid bounds
        gx = min(int(rx), grid_w - 1)
        gy = min(int(ry), grid_h - 1)
        if 0 <= gx < grid_w and 0 <= gy < grid_h:
            density_grid[gy][gx] += 1

    # Each cell is 1m x 1m, so density = count per cell
    max_density = float(np.max(density_grid))
    occupied_cells = np.sum(density_grid > 0)
    total_persons = len(person_positions)
    mean_density = total_persons / (grid_w * grid_h) if (grid_w * grid_h) > 0 else 0

    return max_density, mean_density

def classify_risk_fruin(max_density):
    """
    Classify risk using Fruin Level of Service thresholds.
    LOS A-C (<=0.72): Low Risk
    LOS D-E (0.72-2.17): Medium Risk
    LOS F (>2.17): High Risk
    """
    if max_density <= 0.72:
        return 'LOW', (0, 200, 0)      # Green
    elif max_density <= 2.17:
        return 'MEDIUM', (0, 165, 255) # Orange
    else:
        return 'HIGH', (0, 0, 255)     # Red

def process_frame(frame):
    """
    Main frame processing function.
    Detects persons, computes density, generates heatmap,
    classifies risk, and returns annotated frame with metrics.
    """
    h, w = frame.shape[:2]
    results = model(frame, verbose=False)

    person_boxes = []
    person_positions = [] # bottom-centre points for density

    for result in results:
        for box in result.bboxes:
            cls_id = int(box.cls[0])
            conf = float(box.conf[0])
            if cls_id == PERSON_CLASS_ID and conf >= CONFIDENCE_THRESHOLD:
                x1, y1, x2, y2 = map(int, box.xyxy[0])

```

```

        person_boxes.append((x1, y1, x2, y2, conf))
        # Bottom-centre as ground contact point
        cx = (x1 + x2) // 2
        person_positions.append((cx, y2))

count = len(person_boxes)

# Draw bounding boxes
for (x1, y1, x2, y2, conf) in person_boxes:
    cv2.rectangle(frame, (x1, y1), (x2, y2), (0, 255, 0), 2)
    label = f'Person {conf:.2f}'
    cv2.putText(frame, label, (x1, y1 - 5),
                cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 1)

# Compute density using Approach 3 (Homography)
# area_width_m and area_height_m correspond to dst_points coverage
max_density, mean_density = compute_density_homography(
    person_positions, area_width_m=10.0, area_height_m=10.0
)

# Risk classification using Fruin LOS
risk_level, risk_color = classify_risk_fruin(max_density)

# Generate heatmap overlay
frame = generate_heatmap(frame, person_positions)

# Annotate frame
overlay_text = [
    f'Count: {count}',
    f'Max Density: {max_density:.2f} p/m2',
    f'Mean Density: {mean_density:.2f} p/m2',
    f'Risk: {risk_level}'
]
for i, text in enumerate(overlay_text):
    cv2.putText(frame, text, (10, 30 + i * 30),
                cv2.FONT_HERSHEY_SIMPLEX, 0.8, risk_color, 2)

# Trigger alert if needed
check_and_send_alert(risk_level, count, max_density)

return frame, count, max_density, mean_density, risk_level

```

A.2 HEATMAP GENERATION MODULE (HEATMAP.PY)

```

import cv2
import numpy as np

def generate_heatmap(frame, person_positions, alpha=0.4):
    """
    Generate and overlay a heatmap on the video frame.
    person_positions: list of (x, y) bottom-centre points
    alpha: transparency of heatmap overlay (0=transparent, 1=opaque)
    """

```

```

"""
h, w = frame.shape[:2]
heat = np.zeros((h, w), dtype=np.float32)

# Add Gaussian kernel at each person's position
kernel_size = max(w, h) // 10
if kernel_size % 2 == 0:
    kernel_size += 1 # Must be odd for Gaussian

for (cx, cy) in person_positions:
    if 0 <= cx < w and 0 <= cy < h:
        heat[cy, cx] += 1.0

# Apply Gaussian blur for smooth density representation
heat = cv2.GaussianBlur(heat, (kernel_size, kernel_size), 0)

# Normalise to [0, 255]
if heat.max() > 0:
    heat = (heat / heat.max() * 255).astype(np.uint8)
else:
    heat = heat.astype(np.uint8)

# Apply colour map
heatmap_colour = cv2.applyColorMap(heat, cv2.COLORMAP_JET)

# Blend with original frame
overlay = cv2.addWeighted(frame, 1 - alpha, heatmap_colour, alpha, 0)
return overlay

```

A.3 ALERT SYSTEM MODULE (ALERTS.PY)

```

import smtplib
import time
from email.mime.multipart import MIMEMultipart
from email.mime.text import MIMEText

# Email configuration
SENDER_EMAIL = 'safetycrowd@gmail.com'
SENDER_PASSWORD = 'your_app_password_here' # Use Gmail App Password
RECIPIENT_EMAILS = [
    'ayushgr2811@gmail.com',
    'n.hemunegill@gmail.com',
    'ayushnainwall135@gmail.com'
]

ALERT_COOLDOWN_SECONDS = 300 # 5 minutes between alerts
_last_alert_time = 0

def send_alert_email(person_count, risk_level, max_density):
    """Send HTML-formatted alert email to all registered recipients."""
    timestamp = time.strftime('%Y-%m-%d %H:%M:%S')
    subject = 'Crowd Risk Alert - Immediate Action Required'

```

```

html_body = f'''
<html><body>
<p>Dear Authority Team,</p>
<p>This is an automated notification from the Smart Crowd Monitoring System.</p>
<h3>ALERT DETAILS</h3>
<ul>
    <li><b>People Count:</b> {person_count}</li>
    <li><b>Risk Level:</b> {risk_level}</li>
    <li><b>Max Density:</b> {max_density:.2f} p/m²</li>
    <li><b>Time Detected:</b> {timestamp}</li>
    <li><b>Monitoring Source:</b> Camera 1</li>
</ul>
<p>The crowd density has exceeded the safe threshold.
Immediate preventive action is recommended.</p>
<p>Regards,<br>Smart Crowd Monitoring System<br>AI-Based Surveillance Unit</p>
<p><i>(Note: This is an auto-generated email. Please do not reply.)</i></p>
</body></html>
'''

msg = MIMEMultipart('alternative')
msg['Subject'] = subject
msg['From'] = SENDER_EMAIL
msg['To'] = ', '.join(RECIPIENT_EMAILS)
msg.attach(MIMEText(html_body, 'html'))

try:
    with smtplib.SMTP('smtp.gmail.com', 587) as server:
        server.starttls()
        server.login(SENDER_EMAIL, SENDER_PASSWORD)
        server.sendmail(SENDER_EMAIL, RECIPIENT_EMAILS, msg.as_string())
    print(f'[ALERT] Email sent at {timestamp}')
except Exception as e:
    print(f'[ALERT ERROR] Failed to send email: {e}')

def check_and_send_alert(risk_level, person_count, max_density):
    """Check if alert conditions are met and send email if cooldown has passed."""
    global _last_alert_time
    current_time = time.time()
    if risk_level == 'HIGH' and (current_time - _last_alert_time) >
ALERT_COOLDOWN_SECONDS:
    send_alert_email(person_count, risk_level, max_density)
    _last_alert_time = current_time

```

A.4 FLASK WEB APPLICATION (APP/APP.PY)

```

from flask import Flask, Response, render_template, jsonify
import cv2
import sys
sys.path.append('..')
from inference.yolo_inference import process_frame

app = Flask(__name__)
camera = cv2.VideoCapture(0) # Use 0 for webcam, or path to video file

```

```

# Shared state for statistics
current_stats = {
    'count': 0,
    'max_density': 0.0,
    'mean_density': 0.0,
    'risk_level': 'LOW'
}

def generate_frames():
    """Generator function for MJPEG video stream."""
    global current_stats
    while True:
        success, frame = camera.read()
        if not success:
            break

        # Process frame through the complete pipeline
        annotated, count, max_d, mean_d, risk = process_frame(frame)

        # Update shared statistics
        current_stats = {
            'count': count,
            'max_density': round(max_d, 2),
            'mean_density': round(mean_d, 2),
            'risk_level': risk
        }

        # Encode and yield frame as JPEG
        ret, buffer = cv2.imencode('.jpg', annotated, [cv2.IMWRITE_JPEG_QUALITY, 80])
        frame_bytes = buffer.tobytes()
        yield (b'--frame\r\n'
              b'Content-Type: image/jpeg\r\n\r\n' + frame_bytes + b'\r\n')

@app.route('/')
def index():
    return render_template('index.html')

@app.route('/video_feed')
def video_feed():
    return Response(generate_frames(),
                    mimetype='multipart/x-mixed-replace; boundary=frame')

@app.route('/stats')
def stats():
    return jsonify(current_stats)

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000, debug=False, threaded=True)

```

A.5 ARDUINO IOT MODULE (AURDINO/CROWD_ALERT.INO)

```

// Arduino Uno sketch for physical crowd alert signalling
// Communicates with Python via USB Serial at 115200 baud
// Commands: '1' = HIGH RISK (red LED + buzzer), '0' = LOW/MED (green LED)

const int GREEN_LED = 8;
const int RED_LED = 9;
const int BUZZER_PIN = 10;

void setup() {
  Serial.begin(115200);
  pinMode(GREEN_LED, OUTPUT);
  pinMode(RED_LED, OUTPUT);
  pinMode(BUZZER_PIN, OUTPUT);
  // Default state: normal (green LED on)
  digitalWrite(GREEN_LED, HIGH);
  digitalWrite(RED_LED, LOW);
  digitalWrite(BUZZER_PIN, LOW);
}

void loop() {
  if (Serial.available() > 0) {
    char cmd = Serial.read();
    if (cmd == '1') {
      // HIGH RISK: activate alarm state
      digitalWrite(GREEN_LED, LOW);
      digitalWrite(RED_LED, HIGH);
      // Pulse buzzer three times
      for (int i = 0; i < 3; i++) {
        digitalWrite(BUZZER_PIN, HIGH);
        delay(300);
        digitalWrite(BUZZER_PIN, LOW);
        delay(200);
      }
    } else if (cmd == '0') {
      // Normal state: deactivate alarm
      digitalWrite(GREEN_LED, HIGH);
      digitalWrite(RED_LED, LOW);
      digitalWrite(BUZZER_PIN, LOW);
    }
  }
}

```